

Revised tier-dispatch design — D2, D3, D4, D5 APPROVED

Status: presented 2026-07-29 ~13:25 EDT. **D2, D3, D4, D5 approved by Tom 2026-07-29 ~21:45 EDT**, conditions: preserve fail-closed behaviour, keep it modular, comprehensive tests, docs stay in sync, follow-on work tracked not hidden. D5 additionally requires a formal task + acceptance criteria before implementation (added as plan Task 19, not started — see that entry). D6, D1, D7 were already settled outside this approval (see their own sections). If implementation surfaces a materially conflicting architectural fact, stop and produce a new decision doc rather than deviating silently. Full approval package: `design-docs/2026-07-29-sandbox-decisions-for-approval.md` (untracked, Tom's copy).

Milestone: M1.13b (issue #66), branch `feature/m1.13b-sandbox-plan`.

Inputs: - Codex audit C12 — `docs/superpowers/evidence/2026-07-29-m1.13b-codex-C12-tier-dispatch-audit.md` (twelve claim verdicts, six required corrections, verdict "do not approve unchanged"). - Workflow `wf_0f831c31-cf1`, four read-only source surveys run at 13:15 against HEAD. C12's baseline was `603e124`, ~290 commits stale; the surveys re-pinned every citation and turned up three things C12 missed (marked **NEW** below).

What the surveys confirmed and what they added

Confirmed at HEAD, unchanged from C12's baseline: all five production raw-spawn line numbers; the three swallowing helpers; both extra swallow sites; `probe_argv` genuinely deleted (one past-tense doc comment remains in `escape_contract.rs:1004`).

NEW-1 — the whole classification path is inert in production. `policy_for_repo` (`sandbox/mod.rs:466-483`) sets `let tier = Tier::Network`; unconditionally at line 469 and never calls `tier_for`. `network_need/tier_for` are exercised only by tests. The "dispatch flip" is literally one line.

NEW-2 — there is no single repo-resolution chokepoint, and mutations bypass the one that exists. `read.rs::resolve_repo` (`handlers/read.rs:51-68`) validates an explicit `?repo= id` against the catalog via `state::resolve_worktree`, but falls back to `current()` with no catalog check when the selector is absent. `commit.rs:46,117`, `branch.rs:53`, and `rebase.rs:32,55` never call `resolve_repo` at all — they read `state::current().0` directly and never re-validate it. A per-operation policy hooked at `resolve_repo` would miss every mutation route.

NEW-3 — RefAbsent inverts polarity. Inside one function, `verify_precondition`, `RefAt` (`planner.rs:546`) and `RefExists` (`:557`) refuse on `None`, while `RefAbsent` (`:567`) treats `None` as satisfied and the gate PASSES. Same for `coordinator.rs:104`: `refuse_if_git_busy` propagates `None` through `?`, so "couldn't check" reads as "not busy." A single mechanical "make it a Result" rewrite would get these backwards.

D1 — the chokepoint returns a newtype, not a mutable

Command

`sandboxed(repo)` (`git_cmd.rs:138-141`) returns a live `tokio::process::Command` holding `git -C <repo>` and **no subcommand**; all five callers append `argv` afterward (`git_cmd.rs:149, :253, :270, :283, :303`). Classification therefore never sees what actually runs.

```
pub(crate) struct SandboxedGit { inner: tokio::process::Command }
// stdin / stdout / stderr / kill_on_drop only – NO arg(), NO args()
// consumed by: output(self) -> io::Result<Output> / spawn(self) ->
io::Result<Child>
pub(crate) fn command_async(policy: &Policy, repo: &Path, args: &[&str]) ->
    SandboxedGit
```

`Argv` is fixed at construction and is the same slice that classified the call. `sandboxed(repo, args)` **alone does not close C10 hazard #1** — a caller can still append, which is exactly what all five callers do today. The newtype does close it.

Call-site cost: three pass `&[&str]` directly; `git_stdout_capped` borrows its `&[String]` as a `Vec<&str>`; `rev_parse` binds its formatted `<rev>^{commit}` first. Add an `argv_boundary.rs` tripwire forbidding `.arg(/.args(` on a `SandboxedGit` — textual, matching the existing enforcement style (`argv_boundary.rs:170-213`).

Satisfies C12 correction #2.

D2 — policy built per operation, keyed on the catalog entry

C12 refuted boot-gate totality (claim 12). NEW-2 makes it worse than C12 stated.

- Signature: `policy_for_repo(repo: &Path)` becomes `policy_for(entry: &RepoEntry, need: NetworkNeed)`. The catalog entry already carries `read_only` and `kind` (`catalog.rs:104-120`); today's signature discards both and does `rw.push(repo.to_path_buf())` unconditionally (`sandbox/mod.rs:471`).
- Introduce ONE resolution function used by every handler, reads and mutations alike, returning a target that owns both path and entry. Mutation handlers move off `state::current().0`. This is the admission point; without it, per-operation policy cannot be total.
- The boot capability gate stays — it fails fast and cheap — but it is a gate, never a policy source.

Satisfies C12 correction #4. **APPROVED 2026-07-29.**

D3 — tier from declared intent; argv sniffing demoted to a tripwire

`network_need` describes itself as "a fail-closed fallback, not the authoritative dispatch" (`sandbox/mod.rs:372-386`) and classifies an **empty argv slice as Local**, which routes to `Strict` (`mod.rs:393-410`). That is latent today only because of NEW-1.

The caller declares `NetworkNeed`. `GitOperation` is a closed enum (`git-vista-protocol/src/plan.rs:153`), so variant-to-need is a match the compiler checks. `network_need(argv)`

survives as a cross-check: declared `Local` but `argv` looks `Remote` → debug panic, release escalate to `Strict` and log.

APPROVED 2026-07-29. LANDED 2026-07-30 (plan Task 8, issue #197). As built:

- `sandbox::network_need_for_operation(&GitOperation) -> NetworkNeed` — exhaustive, no wildcard arm. Exactly one variant (`PushBranch`) declares `Remote`.
- The value is threaded from `planner::execute`'s match — the only point where the operation's identity is in scope — through all fifteen `exec_*` functions and the shared runners (`run_git/git/worktree_dirty/run_branch_cmd`) into `git_cmd::git_output_for` → `sandbox::policy_for`. Threading rather than re-deriving is what keeps the exhaustive match in the live data path; a version that computed the need and dropped it would leave the compile-time guarantee empty.
- Call sites with no `GitOperation` (`handlers/read.rs`, `coordinator.rs`, `durable.rs`, `git_cmd`'s `rev_parse/is_ancestor/git_ref_exists/git_ok`) declare `Local` at their own seam — a truthful declaration, not an `argv` fallback; every one of them is a local read or ref write by inspection.
- `sandbox::reconcile_need(declared, argv)` is the cross-check. It is a **checked identity**: it returns `declared`, always. On the one disagreement worth acting on (declared `Local`, `argv` looks `Remote`) it `debug_assert!`s and logs. Keeping `Local` is "escalate to the stricter tier" — `tier_for(Local, false)` is `Strict`, which has no network at all, strictly stricter than the `Network` tier the `argv` argued for. The reverse direction is deliberately not acted on: `REMOTE_SUBCOMMANDS` is incomplete by construction, so narrowing a declared `Remote` would break working pushes on the word of a list that admits it is not authoritative.
- `tier_for`'s `trusted` argument gets its first production source: `sandbox::trust::is_trusted`, keyed on the **canonicalised** repository path (`sandbox::repo_is_trusted`; a canonicalisation failure means untrusted).
- `INV-13/D6` is enforced at policy-construction time, not only at boot: `ShimError::StrictUnavailable { missing }` refuses when `Strict` is selected and the host cannot supply it. No degrade to `Network`, no degrade-and-block-hooks.

D4 — clone gets its own policy constructor

Clone is not a `GitOperation` (confirmed: no `Clone` variant in `plan.rs:153`), its destination does not exist at policy time, and it spawns raw git at `handlers/clone.rs:110-117`. It already extends `Catalog.roots` with the clones root before spawning (`clone.rs:92`) and registers the result afterward (`clone.rs:164`).

```
policy_for_clone(clones_root: &Path) -> Result<Policy, ShimError>
```

RW granted on the **clones root**, not on any repo. `trusted = false` always — no persisted-trust lookup, no operator override. `need = Remote` → `Network` tier, so the `--ro /run` DNS fix applies here. Clone is the one operation that fetches attacker-chosen content, so it is the one operation that must be unreachable at `Unsandboxed`.

Satisfies C12 correction #3. **APPROVED 2026-07-29** (interim already in flight — `clone.rs` now routes through `policy_for_repo(&clones_root())`; the dedicated `policy_for_clone` constructor above refines that, does not gate it).

D5 — execution-unavailable propagates as its own value

`ShimError` (`sandbox/shim.rs:17-28`) already exists and its own doc comment already cites INV-13. It is erased in three steps: `.map_err(|e| e.to_string())?` in `sandboxed`, then `.ok()?` in `rev_parse`, then callers read `None` as fact.

- `rev_parse -> Result<Option<String>, ExecUnavailable>`
- `is_ancestor, git_ref_exists -> Result<bool, ExecUnavailable>`

23 call sites recompile: 20 production plus 3 in `planner/coordination_suite.rs`.

Handling is **not** mechanical — see NEW-3. Three buckets:

Bucket	Sites	On Err
Gate	<code>preconditions</code> (<code>planner.rs:546, :557, :567</code>), <code>CAS pin</code> (<code>commit.rs:118</code>), <code>seed check</code> (<code>planner.rs:1719</code>), <code>busy preflight</code> (<code>coordinator.rs:102</code>), <code>id resolution</code> (<code>activity.rs:346, planner.rs:262</code>)	Refuse with a distinct status. Reuse the existing <code>couldnt_run()</code> + 500 (<code>planner.rs:1047</code> , already 7 call sites). Never 400 — today <code>commit.rs:119</code> and <code>planner.rs:263</code> tell the user "no such branch" / "not a valid object name" when the truth is "git could not run."
Plan observation	<code>head_tip, branch_tip, status</code> (<code>planner.rs:319-323, :497-499</code>)	Carry an explicit <code>Unknown</code> , distinct from <code>Absent</code> . The generation token must hash <code>Unknown</code> differently from empty, or repeated failures compare equal and the freshness gate passes (<code>planner.rs:512-532</code>).
Journal metadata	<code>post-success tips</code> (<code>planner.rs:1129, :1172, :1380, :1405, :1442, :1509, :1651</code>)	Record <code>Unknown</code> , never <code>None-as-absent</code> . Merge and rebase compare two <code>Options</code> (<code>:1405, :1509</code>), so two <code>Unknown</code> s must not report "Already up to date."

Satisfies C12 correction #5, and implements the session decision that the planner needs one explicit "unknown observation" posture rather than site-by-site defaults — 20 individually-reasoned sites is 20 chances to relaunch unknown into fact.

Direction **APPROVED 2026-07-29 — implementation gated on plan Task 19**

(formal task + acceptance criteria, required before any of the 23 sites are touched; see that entry). Recommended sequencing: after D2/D3 land, not concurrent with them.

D6 — INV-13 hard-fail, unchanged

Per ADR 0029 (`docs/adr/0029-strict-tier-hard-fail-when-unavailable.md`). Strict selected with `bwrap/users` absent → the operation refuses. No degrade to Network, no degrade-and-block-hooks. Task 16's `hook_policy_for_repo` maps `CapabilityAbsent` to `HookPolicy::Blocked` — the rejected posture; Task 9 must fix it.

Satisfies C12 correction #1.

D7 — Task 6 completes before the Task 8 dispatch flip

The flip is one line (`sandbox/mod.rs:469`). Five production sites still spawn raw git, re-confirmed at exactly the audited lines:

Site	Function	Confirmed
<code>planner.rs:1037</code>	<code>run_git</code>	yes
<code>handlers/clone.rs:110</code>	<code>clone_repo</code>	yes
<code>durable.rs:549</code>	<code>write_recovery_ref</code>	yes
<code>coordinator.rs:118</code>	<code>absolute_git_dir</code>	yes
<code>handlers/read.rs:901</code>	<code>worktree_status</code>	yes

(`durable.rs:583` is `#[cfg(test)]` — the census is five, not six.)

Flipping dispatch before the migration pays 100% of the measured +17-24 ms per spawn on the interactive read path while clone and every mutation stay unprotected. Separately, `command_sync` has **no production caller** — `#[cfg_attr(not(test), allow(dead_code))]` (`sandbox/spawn.rs:69-75`). Task 6 either wires the blocking sites through it or deletes it. C12 correction #6 (delete `probe_argv`) is already satisfied — it is gone from the tree.

Honest cost

D1 is small. **D2 is the large one** — a new resolution function threaded through the read path plus three mutation handlers that currently bypass it entirely. **D5 is 23 call sites needing per-site judgment**, not a mechanical rewrite, and NEW-3 proves why.

C12 was right: the "already broken today" diagnosis survives, the "therefore near-zero-cost plumbing" conclusion does not.

Out of scope for this milestone

SSH carve-out — issue #188, deferred by Tom on 2026-07-29.

Signed: thomas2025 · 2026-07-29T13:35:00-04:00